

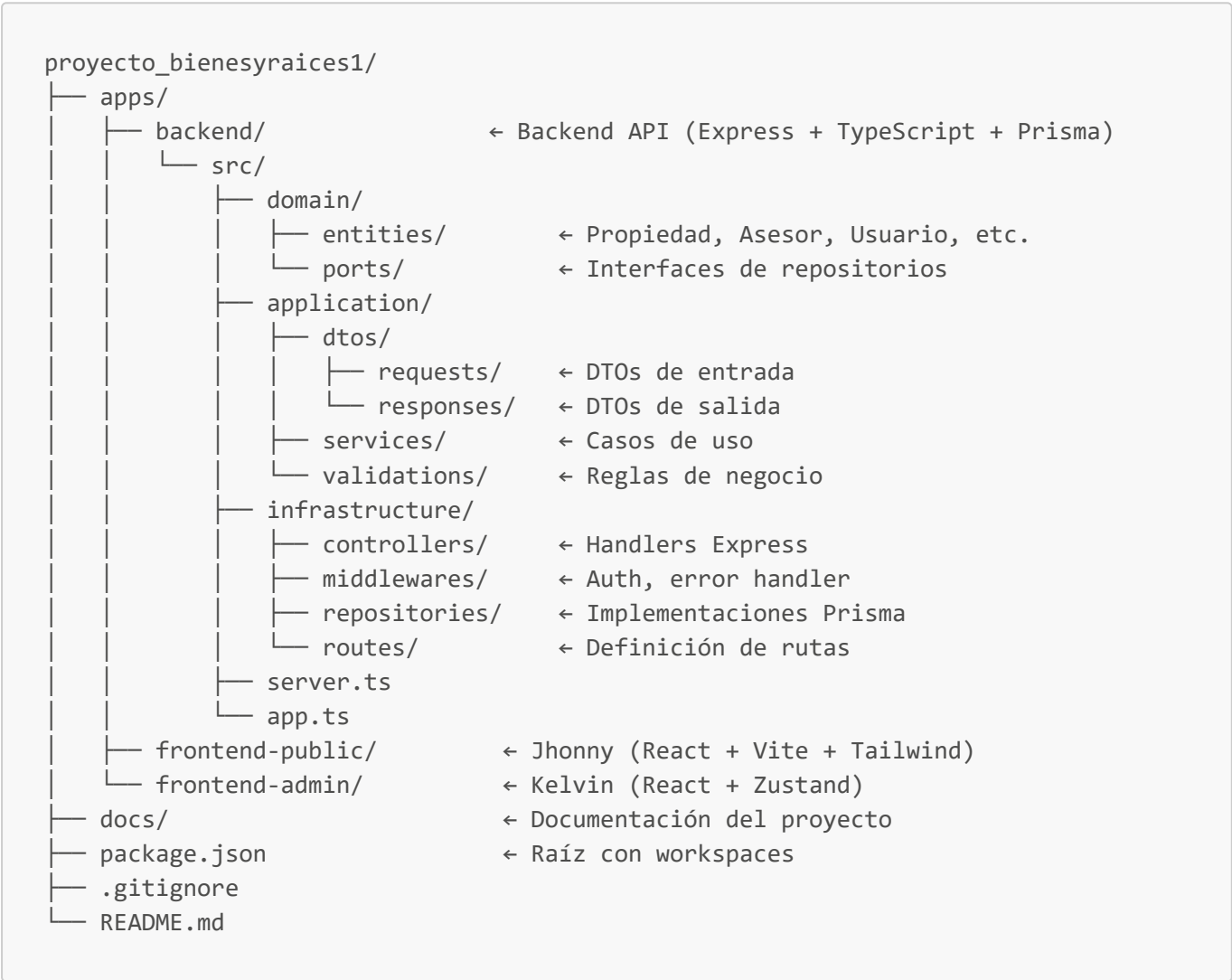
TT-001: Configuración del Monorepo

Responsable: Yandri Alcivar **Sprint:** 1 — Primera Impresión y Navegación **Estimación:** 3 puntos **Estado:** ✔
Completado

Objetivo

Crear la estructura base del proyecto (monorepo) con TypeScript, Express y las herramientas necesarias para que todo el equipo pueda comenzar a trabajar en paralelo.

Estructura de carpetas creada



Tecnologías instaladas

Backend (apps/backend/)

Paquete	Versión	Tipo	Propósito
<code>express</code>	<code>^4.21</code>	Producción	Servidor HTTP
<code>cors</code>	<code>^2.8</code>	Producción	Permitir peticiones del frontend
<code>dotenv</code>	<code>^16.4</code>	Producción	Variables de entorno (.env)
<code>typescript</code>	<code>^6.0</code>	Desarrollo	Tipado estático
<code>tsx</code>	<code>^4.22</code>	Desarrollo	Ejecutar TypeScript directo
<code>@types/node</code>	<code>^26.0</code>	Desarrollo	Tipados de Node.js
<code>@types/express</code>	<code>^5.0</code>	Desarrollo	Tipados de Express
<code>@types/cors</code>	<code>^2.8</code>	Desarrollo	Tipados de cors

Archivos clave creados

`package.json` raíz (workspaces)

```
{
  "name": "proyecto-bienesraices1",
  "private": true,
  "workspaces": ["apps/*"]
}
```

Permite que cada subproyecto en `apps/` tenga sus propias dependencias pero compartan el `node_modules` raíz.

`.gitignore`

```
node_modules/
.env
dist/
.next/
.DS_Store
*.log
```

`apps/backend/src/server.ts`

```
import dotenv from 'dotenv'
dotenv.config()

import express from 'express'
import cors from 'cors'
```

```
const app = express()
const PORT = process.env.PORT || 3000

app.use(cors())
app.use(express.json())

app.get('/api/health', (_req, res) => {
  res.json({ status: 'ok', message: 'API Alpha Inmobiliaria funcionando' })
})

app.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`)
})

export default app
```

apps/backend/package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "scripts": {
    "dev": "tsx watch src/server.ts"
  },
  "dependencies": {
    "express": "^4.21.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.0"
  },
  "devDependencies": {
    "typescript": "^6.0.3",
    "tsx": "^4.22.4",
    "@types/node": "^26.0.1",
    "@types/express": "^5.0.6",
    "@types/cors": "^2.8.19"
  }
}
```

Comandos ejecutados

```
# 1. Crear estructura de carpetas
New-Item -ItemType Directory -Path "apps\backend\src\domain\entities" -Force
New-Item -ItemType Directory -Path "apps\backend\src\domain\ports" -Force
New-Item -ItemType Directory -Path "apps\backend\src\application\dtos\requests" -
Force
New-Item -ItemType Directory -Path "apps\backend\src\application\dtos\responses" -
Force
New-Item -ItemType Directory -Path "apps\backend\src\application\services" -Force
```

```
New-Item -ItemType Directory -Path "apps\backend\src\application\validations" -
Force
New-Item -ItemType Directory -Path "apps\backend\src\infrastructure\controllers" -
Force
New-Item -ItemType Directory -Path "apps\backend\src\infrastructure\middlewares" -
Force
New-Item -ItemType Directory -Path "apps\backend\src\infrastructure\repositories"
-Force
New-Item -ItemType Directory -Path "apps\backend\src\infrastructure\routes" -Force
New-Item -ItemType Directory -Path "apps\frontend-public" -Force
New-Item -ItemType Directory -Path "apps\frontend-admin" -Force
New-Item -ItemType Directory -Path "docs" -Force

# 2. Inicializar package.json del backend
cd apps/backend
npm init -y

# 3. Instalar dependencias
npm install -D typescript tsx @types/node
npm install express cors dotenv
npm install -D @types/express @types/cors

# 4. Push a GitHub
git init
git add .
git commit -m "TT-001: configuración inicial del monorepo"
git branch -M main
git remote add origin https://github.com/chaoslov/proyecto_bienesyraices1.git
git push -u origin main
```

Verificación

```
cd apps/backend
npm run dev
```

Salida esperada:

```
Servidor corriendo en http://localhost:3000
```

Endpoint de salud: GET /api/health → {"status":"ok","message":"API Alpha Inmobiliaria funcionando"}

Arquitectura aplicada

Se sigue el patrón de **Arquitectura Hexagonal** (Puertos y Adaptadores) con la siguiente separación de capas:

- **Domain:** Entidades y puertos (interfaces de repositorio) — núcleo del negocio
 - **Application:** DTOs, servicios (casos de uso) y validaciones
 - **Infrastructure:** Controladores, rutas, middlewares e implementaciones concretas de repositorios
-

Notas

- El proyecto usa **npm workspaces** para gestionar los subproyectos del monorepo
- El nombre del paquete raíz en `package.json` usa guiones: `proyecto-bienesraices1` (no coincide exactamente con el nombre del repo en GitHub, pero no afecta)
- Los archivos `server.ts` se ejecutan con `tsx` en modo watch para desarrollo
- Se configuró `.gitignore` para excluir `node_modules/`, `.env` y archivos de compilación